

MAT120 : Mathématiques discrètes

Thomas Brüstle

Automne 2026

Table des matières

1	Logique	3
1.1	Propositions et compositions élémentaires	3
1.2	Tables de vérité	3
1.3	Conditionnelle et bi-conditionnelle	5
1.4	Quelques tautologies importantes	8
1.5	Predicats et quantificateurs	9
1.6	Voir : Lois de la logique du premier ordre sur Moodle.	11
1.7	Exercices de vérification	11
2	Techniques de preuve	14
2.1	Avant de prouver : partir des définitions	14
2.2	Preuve directe	14
2.3	Preuve par contraposée	15
2.4	Prouver une équivalence	16
2.5	Preuve par contradiction	16
2.6	Preuve par disjonction de cas	18
2.7	Réfuter un énoncé : le contre-exemple	18
2.8	Preuves d'existence et d'unicité	19
2.9	Preuve par récurrence	19
2.10	Le principe des tiroirs	20
2.11	Comment choisir une méthode ?	21
2.12	Conseils de rédaction	22
2.13	Exercices	22
3	Introduction à Lean	25
3.1	Pourquoi utiliser un assistant de preuve ?	25
3.2	La version de Lean utilisée dans ce cours	26
3.3	Comment Lean voit une démonstration	26

3.4	Premier exemple : démontrer une implication	28
3.5	Le dictionnaire logique de base	28
3.6	Quelques exemples de logique en Lean	29

3 Introduction à Lean

Dans les deux premiers chapitres, nous avons appris à reconnaître la forme logique d'un énoncé et à choisir une méthode de preuve adaptée. Nous allons maintenant reprendre plusieurs de ces idées sous un angle nouveau : nous allons demander à un ordinateur de *vérifier* nos démonstrations.

Le logiciel utilisé sera *Lean*, un assistant de preuve interactif. Lean n'est pas un logiciel qui remplace la réflexion mathématique. Il oblige au contraire à préciser les objets, les hypothèses et chaque étape logique avec suffisamment de soin pour qu'un programme puisse les vérifier.

« J'ai appris l'algèbre linéaire trois fois : une première fois comme étudiant, une deuxième fois comme chargé d'exercices en l'enseignant à d'autres étudiants, et une troisième fois lorsque je l'ai implémentée sur ordinateur. À chaque fois, ma connaissance et ma compréhension se sont approfondies. »

C'est dans le même esprit que nous introduisons Lean dans ce cours. Nous n'allons pas apprendre beaucoup de nouvelles mathématiques dans ce chapitre. Nous allons plutôt reprendre des principes de logique et des techniques de preuve déjà rencontrés et les exprimer de façon assez précise pour qu'ils puissent être vérifiés par Lean. L'objectif est que cette nouvelle façon d'écrire les démonstrations approfondisse notre compréhension de ce que nous faisons déjà sur papier.

3.1 Pourquoi utiliser un assistant de preuve ?

Une preuve écrite sur papier est destinée à un lecteur humain. Nous omettons souvent des étapes que nous jugeons évidentes. Nous utilisons des expressions comme «donc», «on voit que», «il suffit de montrer» ou «par hypothèse», et le lecteur reconstruit une partie du raisonnement.

Un assistant de preuve demande davantage de précision. Il faut spécifier :

- quels sont les objets mathématiques en jeu ;
- quelles sont les hypothèses disponibles ;
- quel est exactement l'énoncé qu'il reste à démontrer ;
- quelle règle logique ou quel résultat déjà établi justifie chaque étape.

Cette contrainte peut sembler lourde au début. Mais elle a un avantage pédagogique important : elle rend visible la structure d'une démonstration.

Lean est aujourd'hui l'un des assistants de preuve de référence pour la formalisation des mathématiques. Sa bibliothèque communautaire, *Mathlib*, contient un très grand nombre de définitions et de théorèmes couvrant de nombreux domaines des mathématiques. Des projets de recherche avancés ont été entièrement formalisés en Lean. Un exemple important est le *Liquid Tensor Experiment*, qui a formalisé un résultat de Clausen–Scholze en géométrie condensée. Le projet a été achevé en 2022.¹

1. Voir <https://leanprover-community.github.io/liquid/>.

Ces exemples sont très loin du niveau de ce cours. Mais le principe logique est le même : une preuve compliquée est construite à partir d'une succession de petites étapes dont chacune doit être justifiée.

3.2 La version de Lean utilisée dans ce cours

Supposons, Montrons que, On conclut par, Par ... on obtient, On
 contrapose.

Tous les exemples et exercices du cours sont préparés dans un Codespace GitHub :

Le Codespace fournit un environnement déjà configuré. Nous n'aurons donc pas besoin de commencer par installer Lean ni par configurer sa bibliothèque.

Pour comprendre Lean, il est utile de retenir trois idées simples.

Dans Lean, une proposition mathématique P est écrite comme un objet de type `Prop`. Par exemple,

2. Voir <https://openai.com/index/navier-stokes-solution/>. À la date de rédaction de ces notes, il s'agit d'une annonce extrêmement récente ; l'évaluation mathématique indépendante et la reconnaissance officielle d'un résultat de cette ampleur sont des questions distinctes de la vérification informatique du fichier Lean.

26

signifie que P est un énoncé auquel on peut demander une preuve.

Une hypothèse

$$h : P$$

se lit : « h est une preuve de P », ou plus simplement : «on sait que P est vrai».

Cette façon de penser est appelée le principe *propositions comme types* (ou correspondance de Curry–Howard). Pour ce cours, il n’est pas nécessaire d’en étudier la théorie. Nous l’utiliserons seulement comme règle de lecture :

$$h : P \quad \text{signifie que } h \text{ justifie } P.$$

3.3.2 Le contexte et le but

Pendant une démonstration, Lean affiche deux sortes d’information :

- le *contexte* : les objets et hypothèses actuellement disponibles ;
- le *but* : ce qu’il reste à démontrer.

Par exemple, l’état

$$\frac{P, Q : \text{Prop}, \quad h_P : P, \quad h : P \rightarrow Q}{\vdash Q}$$

se lit ainsi : nous disposons de deux propositions P, Q , d’une preuve de P et d’une preuve de $P \rightarrow Q$; il reste à démontrer Q .

Le symbole $\vdash$ peut se lire «il reste à prouver».

3.3.3 Une commande transforme le problème courant

Une commande Lean fait essentiellement l’une des deux choses suivantes :

1. elle termine le but courant en fournissant une justification ;
2. elle transforme le but courant en un ou plusieurs buts plus simples.

Nous pouvons schématiser une démonstration ainsi :

$$\text{contexte} + \text{but} \longrightarrow \text{commande} \longrightarrow \text{nouveau contexte} + \text{nouveau(x) but(s)} \longrightarrow \cdots \longrightarrow \text{aucun but}$$

Quand il ne reste plus aucun but, la démonstration est terminée.

Derrière ces commandes, Lean construit un objet formel appelé un *terme de preuve*. Les commandes que nous utilisons servent donc à construire la preuve ; le noyau de Lean vérifie ensuite que le terme obtenu a bien le type annoncé. Autrement dit, les commandes peuvent nous aider, mais la vérification finale est effectuée indépendamment par le noyau.

3.4 Premier exemple : démontrer une implication

Reprenons l’une des règles les plus importantes du chapitre 1. Pour démontrer

$$P \rightarrow Q,$$

on suppose P et on cherche à démontrer Q .

Dans notre version de Lean, un exemple minimal a la forme suivante.

```
-- Prouver une implication : ‘Supposons’  
Exemple "Prouver  $P \rightarrow Q$  : Supposons"  
  Données : (P Q : Prop)  
  Hypothèses : (h : P → Q)  
  Conclusion : P → Q  
Démonstration :  
  Supposons hP : P  
  On conclut par h hP  
QED
```

Regardons les deux lignes de démonstration.

Étape 1. Le but est d’abord $P \rightarrow Q$. La commande

Supposons hP : P

introduit l’hypothèse $h_P : P$. Le nouveau but est Q .

Étape 2. Nous disposons maintenant de

$$h : P \rightarrow Q \quad \text{et} \quad h_P : P.$$

Par modus ponens, nous obtenons Q . La ligne

On conclut par h hP

exprime exactement ce raisonnement.

Il n’y a donc pas deux logiques différentes, une logique «sur papier» et une logique «dans Lean». Lean nous force simplement à rendre explicite la règle logique utilisée.

Remarque 3.2 (Pourquoi l’implication ressemble-t-elle à une fonction?). On peut voir une preuve de $P \rightarrow Q$ comme une procédure qui transforme une preuve de P en une preuve de Q . C’est pourquoi Lean utilise le même symbole $\rightarrow$ pour une implication et pour le type d’une fonction. Dans l’exemple précédent, h transforme $h_P : P$ en $h h_P : Q$.

3.5 Le dictionnaire logique de base

Les commandes de Verbose Lean reflètent directement les règles de logique vues au chapitre 1. Le tableau suivant donne les premières correspondances à retenir.

Forme logique	Sur papier	Idée en Verbose Lean
$P \rightarrow Q$ à prouver	Supposer P , puis montrer Q .	Supposons <code>hP : P</code>
$h : P \rightarrow Q$ à utiliser	À partir de P , déduire Q .	On conclut par <code>h hP</code>
$P \wedge Q$ à utiliser	Extraire les deux informations P et Q .	Par <code>h</code> on obtient <code>hP hQ</code>
$P \wedge Q$ à prouver	Prouver séparément P et Q .	Montrons d'abord que P , puis Montrons maintenant que Q
$P \vee Q$ à utiliser	Faire deux cas : P , puis Q .	On discute en utilisant <code>h</code>
$P \vee Q$ à prouver	Choisir une branche et la démontrer.	Montrons que P ou Montrons que Q
$P \leftrightarrow Q$ à prouver	Prouver les deux implications.	Montrons d'abord que $P \rightarrow Q$, puis Montrons maintenant que $Q \rightarrow P$
$\forall x, P(x)$ à prouver	Prendre x arbitraire.	Soit <code>x</code>
$\exists x, P(x)$ à prouver	Fournir un témoin x_0 .	Montrons que <code>x0</code> convient
$\exists x, P(x)$ à utiliser	Choisir un témoin fourni par l'hypothèse.	Par <code>h</code> on obtient <code>x0</code> tel que <code>hx0</code>
Contraposée	Remplacer $P \rightarrow Q$ par $\neg Q \rightarrow \neg P$.	On contrapose
Contradiction	Supposer la négation du but et obtenir <code>false</code> .	Supposons par l'absurde <code>h : ¬P</code>

Ce tableau n'est pas une liste de «trucs informatiques». Chaque ligne est simplement une règle logique déjà rencontrée dans les deux premiers chapitres.

3.6 Quelques exemples de logique en Lean

3.6.1 Le modus ponens

Sur papier, la règle est

$$P, \quad P \rightarrow Q \quad \Longrightarrow \quad Q.$$

En Lean :

```
Exemple "Modus ponens"
Données : (P Q : Prop)
Hypothèses : (hP : P) (hPQ : P → Q)
Conclusion : Q
```

```
Démonstration :  
  On conclut par hPQ hP  
QED
```

La preuve est courte parce que toute la structure logique est déjà visible dans les types $h_P : P$ et $h_{PQ} : P \rightarrow Q$.

3.6.2 Une conjonction

Supposons que P soit vrai et que P implique Q . Nous voulons montrer $P \wedge Q$.

```
Exemple "Construire une conjonction"  
Données : (P Q : Prop)  
Hypothèses : (hP : P) (hPQ : P → Q)  
Conclusion : P ∧ Q  
Démonstration :  
  Montrons d'abord que P  
  On conclut par hP  
  Montrons maintenant que Q  
  On conclut par hPQ hP  
QED
```

Les commandes `Montrons d'abord que P` et `Montrons maintenant que Q` séparent le but $P \wedge Q$ en deux buts : d'abord P , puis Q . Nous retrouvons exactement la règle du chapitre 1 : pour démontrer une conjonction, il faut démontrer les deux composantes.

3.6.3 Un quantificateur universel

Pour démontrer

$$\forall x \in \mathbb{R}, \quad x^2 = (-x)^2,$$

on choisit un réel arbitraire x puis on effectue le calcul.

La structure de la preuve est :

```
Démonstration :  
  Soit x : ℝ  
  On calcule  
QED
```

La commande `Soit x : ℝ` est la traduction directe de la phrase que nous écrivons sur papier : «Soit $x \in \mathbb{R}$ arbitraire.»

3.6.4 Un quantificateur existentiel

Pour montrer qu'il existe un entier naturel k tel que

$$8 = 2k,$$

il suffit de proposer le témoin $k = 4$.

Démonstration :

Montrons que 4 convient

On calcule

QED

La logique est exactement celle du chapitre 1 :

$\exists k P(k)$ se démontre en exhibant un témoin k_0 et en prouvant $P(k_0)$.